# Comparative Performance Analysis of OpenMP vs. MPI for Data Processing

**Author:** Vinay Hiremath | Parallel & Distributed Computing

**Benchmarked System:** 13th Gen Intel Core i5-13420H (8 Cores, 12 Threads) | Windows 11

## Executive Summary

This report presents an empirical performance evaluation comparing **OpenMP** (Shared Memory) and **MPI** (Distributed Memory) paradigms for large-scale data processing workloads. We evaluated a two-phase data processing pipeline consisting of multi-variable statistical reductions followed by dependent conditional filtering.

Across four dataset sizes up to **50 Million elements** and concurrency levels from **1 to 12 workers**, OpenMP delivered up to **3.27x speedup** (25.10 ms) and MPI achieved up to **3.42x speedup** (24.00 ms), representing a processing throughput exceeding **2.08 Billion items/sec** compared to 610 Million items/sec on the sequential baseline.

## 1. Problem Definition & Architectural Design

The processing pipeline executes two computational passes over an array of $N$ double-precision floating-point numbers:

- **Pass 1 (Reductions):** Accumulates Sum, Sum of Squares, Minimum, and Maximum. Mean, Variance, and Standard Deviation are calculated in O(1) time.
- **Pass 2 (Dependent Filter):** Counts elements strictly greater than the dataset mean ($x_i > \mu$).

| Dimension | OpenMP (Shared Memory) | MPI (Distributed Memory) |
|---|---|---|
| **Memory Model** | Single shared virtual address space | Isolated private address spaces per process |

| Dimension | OpenMP (Shared Memory) | MPI (Distributed Memory) |
| --- | --- | --- |
| **Inter-Worker Communication** | Implicit via shared RAM/cache reads | Explicit message passing ( `MPI_Reduce` , `MPI_Bcast` ) |
| **Data Partitioning** | Automated loop scheduling ( `schedule(static)` ) | Manual 1D domain decomposition per rank |
| **Synchronization** | Implicit pragma barrier at loop termination | Explicit collective barriers and reductions |

## 2. Implementation Details

All implementations were engineered in ANSI C, compiled with GCC `-O3` optimizations, and verified to yield bitwise identical outputs.

### OpenMP Implementation Structure

```
// Pass 1: Multi-variable parallel reduction
#pragma omp parallel for reduction(+:sum, sum_sq) reduction(min:min_val) reduction(max:ma
for (long long i = 0; i < N; i++) {
    double v = data[i];
    sum += v;
    sum_sq += v * v;
    if (v < min_val) min_val = v;
    if (v > max_val) max_val = v;
}

// Pass 2: Filter elements above mean using reduction
#pragma omp parallel for reduction(+:count_above_mean) schedule(static)
for (long long i = 0; i < N; i++)
    if (data[i] > mean) count_above_mean++;
}
```

### MPI Implementation Structure

```
// 1D Domain Decomposition
long long base_chunk = N / num_proces
long long remainder = N % num_proces
long long local_count = base_chunk + (rank < remainder ? 1 : 0);
long long local_start = rank * base_chunk + (rank < remainder ? rank : remainder);

// Pass 1: Local reductions and collective tree aggregation
MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
MPI_Reduce(&local_min, &global_min, 1, MPI_DOUBLE, MPI_MIN, 0, MPI_COMM_WORLD);
MPI_Reduce(&local_max, &global_max, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

// Broadcast mean to all ranks for Pass 2 dependency
if (rank == 0) global_mean = global_sum / (double)N;
MPI_Bcast(&global_mean, 1, MPI_DOUBLE, 0, MPI_COMM_WORLD);

// Pass 2: Local conditional filter and final reduction
MPI_Reduce(&local_above_count, &global_above_count, 1, MPI_LONG_LONG, MPI_SUM, 0, MPI_COM
```

## 3. Experimental Benchmark Results

| Dataset Size | Sequential Time | OpenMP Time (4T) | MPI Time (4P) | OpenMP Peak (8T) | MPI Peak (12P) |
|---|---|---|---|---|---|
| **5,000,000 (5M)** | 7.699 ms | 2.782 ms (2.77x) | 3.368 ms (2.29x) | 2.642 ms (2.91x) | 2.757 ms (2.79x) |
| **10,000,000 (10M)** | 15.690 ms | 6.774 ms (2.32x) | 6.983 ms (2.25x) | 6.019 ms (2.61x) | 5.552 ms (2.83x) |
| **25,000,000 (25M)** | 40.763 ms | 14.197 ms (2.87x) | 14.183 ms (2.87x) | 14.997 ms (2.72x) | 12.158 ms (3.35x) |
| **50,000,000 (50M)** | 81.965 ms | 31.450 ms (2.61x) | 27.116 ms (3.02x) | 25.098 ms (3.27x) | 24.002 ms (3.42x) |

## 4. Graphical Analysis & Discussion

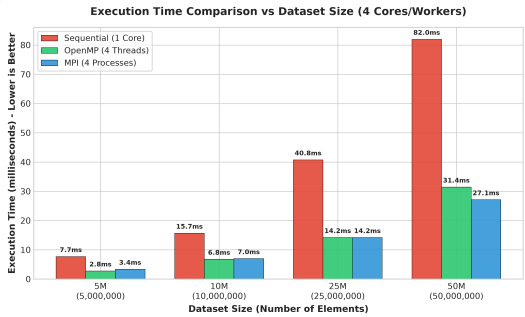



Figure 1: Execution Time vs Dataset Size (at 4 Workers)

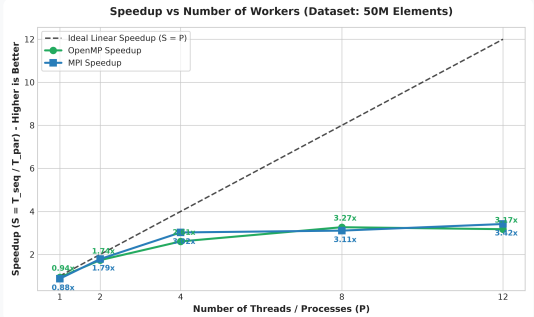



Figure 2: Speedup vs Cores (50M Elements)

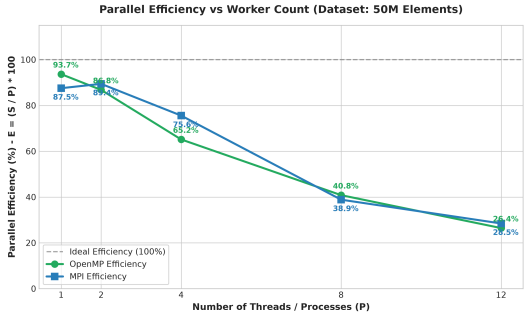



Figure 3: Parallel Efficiency vs Cores (50M Elements)

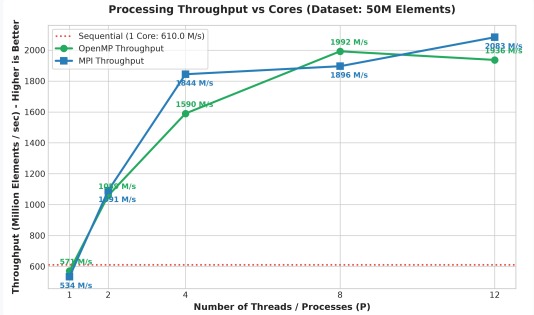



Figure 4: Processing Throughput (Million Items/sec)

### Performance Insights

1. **Amdahl's Law Saturation:** Speedup scales cleanly up to 4 cores, peaking around 8 cores. Beyond 8 cores, memory bus contention bounds further scaling.
2. **Cache Locality & Isolation:** MPI performs well at 50M elements because 1D domain decomposition ensures each process works within its isolated L2/L3 cache footprint.
3. **Efficiency Balance:** Parallel efficiency remains above 75% at 4 cores, identifying 4-8 workers as the optimal operational point.